

Evaluación Parcial 3

Encargo con Defensa Técnica

Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
DSY1103	Desarrollo FullStack 1	2semanas/15min	25%

1. Situación Evaluativa 1:

Ejecución práctica	X	Entrega de encargo	X	Presentación / Defensa Técnica		Prueba escrita (formato quiz)
--------------------	---	--------------------	---	--------------------------------	--	-------------------------------

2. Instrucciones generales

Descripción
- La Evaluación Sumativa 3 corresponde al cierre técnico formal del Proyecto Semestral de Arquitectura de Microservicios, el cual será desarrollado en equipos de 2 a 3 integrantes y tiene como objetivo validar, documentar y desplegar una solución completa de los desarrollado en la Sumativa 2. - Cada equipo deberá mantener y perfeccionar los microservicios construidos en las evaluaciones anteriores, integrando ahora pruebas unitarias, documentación técnica, comunicación distribuida mediante API Gateway y despliegue en entornos locales o remotos, asegurando calidad, trazabilidad y operatividad del sistema. Como mínimo, se deben mantener al menos 10 microservicios por equipo. pudiendo ampliarse si la arquitectura lo requiere, siempre que se preserve la coherencia, cohesión y separación funcional entre servicios.

- El tiempo asignado para el desarrollo de esta evaluación en el **taller de alto cómputo** es de **cuatro sesiones pedagógicas**.
- El encargo deberá ser **entregado en la semana 15**, antes de la primera sesión de evaluación. La **presentación/defensa técnica se realizará en las semanas 15, 16 o 17 de acuerdo a la planificación del Docente**, con una duración máxima de **15 minutos por estudiante**. La defensa técnica se organizará **al azar**, por lo que todos los equipos deben estar preparados desde el inicio del período de evaluación.
- Aunque el proyecto se trabaja en **grupos**, la **defensa es individual**. Cada estudiante será evaluado de forma personal, en base a su **desempeño, claridad y dominio del proyecto**. La nota dependerá exclusivamente de lo que cada uno demuestre en la defensa técnica, por lo que es fundamental conocer a fondo todos los aspectos del desarrollo.
- Si durante la defensa un/a estudiante **no puede explicar con claridad el proyecto, no logra ejecutar correctamente la aplicación o no sabe modificar el código de forma funcional**, se asumirá que **no participó activamente en el desarrollo**.
- Esto afectará directamente su calificación, que será asignada **según su propio desempeño, independientemente de que el proyecto esté bien construido** o que su compañero/a lo haya defendido correctamente.
- Informar a los/las estudiantes que, al ser un encargo, deberán realizarlo previamente en su tiempo de trabajo autónomo

Propósito de la evaluación:

El propósito de esta evaluación es verificar la capacidad del equipo y de cada estudiante para validar, documentar, explicar y desplegar una arquitectura distribuida basada en microservicios independientes, con comunicación REST, pruebas unitarias, documentación formal y despliegue real, demostrando dominio en:

- La estructuración de proyectos bajo el patrón CSR (Controller–Service–Repository/Model).
- La implementación y explicación de reglas de negocio, validaciones y relaciones de entidad.
- La integración de comunicación REST entre microservicios, garantizando interoperabilidad y manejo adecuado de errores.
- El desarrollo y comprensión de pruebas unitarias para validar reglas de negocio y funcionalidad del microservicio.
- La elaboración y explicación de documentación técnica utilizando Swagger/OpenAPI.
- La configuración y uso de un API Gateway para centralizar y administrar el enrutamiento entre microservicios.
- La configuración de archivos YAML para gestionar propiedades de entorno, rutas, perfiles y parámetros de despliegue.
- La configuración y despliegue de microservicios en entornos locales o remotos (Docker, Railway, Render u otros), asegurando su operatividad.

- El uso de herramientas colaborativas y control de versiones como GitHub y Trello para gestionar avances, roles y evidencia de trabajo.

Instrucciones específicas de la Evaluación:

Consideraciones para la realización de la ejecución práctica:

Se recomienda realizar esta evaluación en laboratorio o sala equipada con:

- Computadores con acceso a IntelliJ IDEA o VS Code, JDK y todas las dependencias necesarias para Spring Boot.
- Posibilidad de ejecutar Docker Desktop o herramientas equivalentes cuando el despliegue se realice mediante contenedores.
- Accesos configurados a plataformas de despliegue remoto como Railway, Render u otras (cuando se utilicen).
- Conexión a internet estable para:
 - Descarga de dependencias Maven/Gradle.
 - Acceso a repositorios GitHub.
 - Acceso a paneles de despliegue remoto (Railway, Render, etc.).
- Herramientas de prueba como Postman u otro cliente REST.
- Posibilidad de compartir pantalla para que el docente pueda visualizar:
 - Código fuente.
 - Ejecución de pruebas unitarias.
 - Documentación Swagger/OpenAPI.
 - Configuración del Gateway y archivos YAML.
 - Despliegue local y/o remoto en funcionamiento.

Requisitos obligatorios de la evaluación:

El desarrollo debe integrar todos los aprendizajes adquiridos durante la Experiencia de Aprendizaje 3, incluyendo los siguientes elementos:

- **Pruebas unitarias con JUnit (y Mockito u otro framework de mocking)**

- Creación de clases de test en src/test/java siguiendo una estructura organizada.
- Implementación de pruebas unitarias sobre servicios y/o lógica de negocio, validando reglas clave del dominio.
- Uso de asserts adecuados y estructura Given–When–Then u otra convención clara.
- Empleo de mocks o dobles de prueba para simular repositorios y dependencias cuando corresponda.
- Ejecución de las pruebas antes de la defensa, asegurando que se encuentren en estado exitoso.
- **Documentación técnica con Swagger/OpenAPI**
 - Configuración de dependencias necesarias (por ejemplo, springdoc-openapi).
 - Habilitación de la UI de Swagger para la exploración de endpoints.
 - Documentación de los endpoints principales, incluyendo:
 - Descripción funcional.
 - Parámetros de entrada.
 - Códigos de respuesta.
 - Ejemplos de respuesta en JSON.
 - Coherencia entre lo documentado y el comportamiento real del microservicio.
- **Comunicación entre microservicios y API Gateway**
 - Consumo de endpoints entre microservicios mediante WebClient o Feign Client.
 - Manejo de respuestas, validación de datos y tratamiento de errores remotos.
 - Configuración de un API Gateway (Spring Cloud Gateway o equivalente) para centralizar rutas hacia los microservicios.
 - Definición de rutas, prefijos y filtros básicos que permitan un flujo controlado de las solicitudes.
- **Configuración con archivos YAML**
 - Uso de archivos .yml o .yaml para definir propiedades de entorno, puertos, rutas del Gateway, perfiles y variables relevantes.
 - Organización clara de perfiles (por ejemplo, dev, prod o equivalentes) cuando aplique.
 - Separación adecuada de configuraciones sensibles o dependientes del entorno.
- **Despliegue local y/o remoto de microservicios**
 - Configuración de ejecución de los servicios de forma local (ya sea desde el IDE o mediante Docker).

- Despliegue en al menos un entorno remoto (ej.: Railway, Render u otra plataforma) cuando sea parte del alcance definido por el docente.
- Verificación de que los servicios se encuentren operativos y accesibles durante la defensa.
- **Buenas prácticas y calidad de código**
 - Estructura de proyecto limpia y coherente con el patrón CSR.
 - Nombres significativos para clases, métodos y variables.
 - Código comentado sólo cuando sea necesario para aclarar lógica compleja.
 - Eliminación de código muerto o duplicado antes de la entrega.
- **Integración de herramientas de trabajo colaborativo y gestión de versiones como Git**

Forma de entrega:

- El proyecto debe estar subido a un repositorio de GitHub público con acceso otorgado al docente y a los miembros del equipo.
- El repositorio debe contener:
 - Carpeta del proyecto o proyectos de microservicios.
 - Archivo README.md con:
 - Descripción del contexto o dominio del proyecto.
 - Nombres de los/las estudiantes.
 - Listado de microservicios implementados.
 - Rutas principales del Gateway (cuando aplique).
 - Enlaces a la documentación Swagger (local o remota).
 - Instrucciones básicas de ejecución local y remota
 - Pruebas unitarias con al menos el 80% de cobertura.
 - Commits bien diseñados, distribuidos y generados que expliquen de manera correcta cada uno de ellos (no usar textos o descripciones no técnicas).
 - **Si se detectan cambios así sean mínimos luego de la fecha de entrega en el repositorio por cualquiera de los integrantes del equipo y antes de su evaluación se les asignará un 1.0 de manera automática en la nota tanto grupal como individual.**

- Se subirá al **enlace de AVA grupal el código fuente desarrollado** (con los mismos elementos que se suben al repositorio) por un solo miembro del equipo. En caso de no entregar en AVA no se podrá realizar por ningún otro medio ni después de la fecha límite y se le generará un 1.0 tanto en la nota grupal como individual de manera automática.
- Se activará el enlace en **AVA individual escribiendo su nombre y apellido y número de equipo asignado**, así como el nombre de su aplicación o contexto. En caso de no activarlo en el plazo establecido no tendrá derecho a la defensa y se le asignará un 1.0 como calificación individual de manera automática (Si podrá acceder a la nota grupal en caso de haberlo activado)

Consideraciones para todos los estudiantes:

- La defensa técnica es individual.
- No se permite el uso de inteligencia artificial, herramientas automáticas de código o asistencia externa durante la sesión.
- El uso de internet debe limitarse estrictamente a la descarga de dependencias del proyecto Spring Boot.
- El/la docente monitoreará continuamente el trabajo del estudiante mediante observación directa o herramientas de control de laboratorio.
- Cualquier indicio de colaboración no autorizada o plagio implicará la anulación inmediata de la evaluación.
- Puedes solicitar repetir la pregunta si no la entiendes.
- Puedes anotar los pasos que vas a seguir.
- El/la docente evaluará tanto lo que haces como lo que explicas.
- Si algún recurso de la entrega del código no está implementado en la aplicación desarrollada, todos los ítems tanto grupales como individuales asociados a dicho elemento se calificarán de manera inmediata con un 0, esto incluye a los elementos donde se deben desarrollar o corregir errores en tiempo real durante la defensa.

3. Pauta de Evaluación Rúbrica

Categoría	% logro	Descripción niveles de logro
Muy buen desempeño	100%	Demuestra un desempeño destacado, evidenciando el logro de todos los aspectos evaluados en el indicador.
Desempeño aceptable	60%	Demuestra un desempeño competente, evidenciando el logro de los elementos básicos del indicador, pero con omisiones, dificultades o errores.
Desempeño incipiente	30%	Presenta importantes omisiones, dificultades o errores en el desempeño, que no permiten evidenciar los elementos básicos del logro del indicador, por lo que no puede ser considerado competente.
Desempeño no logrado	0%	Presenta ausencia o incorrecto desempeño.

Indicador de Evaluación	Categorías de Respuesta				Ponderación Indicador de Evaluación	
	Muy buen desempeño 100%	Desempeño aceptable 60%	Desempeño incipiente 30%	Desempeño no logrado 0%		
Dimensión Entrega de Encargo (Grupal)						
IE 1.2.1 Estructura el microservicio aplicando el patrón CSR (Controller–Service–Repository/Model), con separación real de responsabilidades y paquetes por capa.	Estructura el proyecto por paquetes (controller, service, model/repository); el controller solo orquesta; el service concentra lógica de negocio; el repository/model gestiona datos; dependencias en dirección correcta.		Estructura el proyecto por paquetes (controller, service, model/repository); pero con algunos errores en implementación de funcionalidades propias de uno o dos paquetes.	Estructura el CSR parcial: existe separación, pero queda lógica en el controller o hay acoplamientos menores entre capas.	No estructura el patrón de diseño	2%
IE 2.2.1 Implementa reglas de negocio, validaciones y relaciones de entidad garantizando	Implementa todas las reglas de negocio necesarias, su lógica es sólida y responde		Implementa la mayoría de reglas con pequeñas omisiones o errores menores.	Implementa reglas básicas, pero omite elementos importantes del dominio o las aplica incorrectamente.	No implementa reglas de negocio o son incorrectas.	3%

Subdirección de Diseño Instruccional

integridad, consistencia de datos y manejo adecuado de flujos críticos del servicio.	correctamente a casos reales del dominio.					
IE 2.4.1 Integra comunicación REST efectiva entre microservicios, gestionando correctamente solicitudes, respuestas, códigos HTTP y manejo de errores	Integra comunicación REST, controla solicitudes, respuestas, timeouts, errores remotos, códigos HTTP adecuados, y garantiza flujo estable entre servicios; datos reconciliados correctamente		Realiza comunicación REST funcional, aunque presenta errores menores de validación, manejo de errores o códigos de respuesta; flujo general operativo.	La comunicación funciona parcialmente o presenta errores frecuentes; manejo deficiente de errores remotos; inconsistencias en parámetros o respuestas.	No integra comunicación REST o el servicio no responde; llamadas incorrectas o inexistentes.	3%
IE 2.5.1 Gestiona un repositorio GitHub ordenado, con commits técnicos, progresivos y distribuidos equitativamente entre los integrantes.	El repositorio presenta commits técnicos, descriptivos, progresivos, correctamente versionados y distribuidos; ramas limpias; historial claro y profesional y equitativos.		Commits presentes, pero con desequilibrio entre integrantes.	Commits escasos, mensajes poco técnicos o desorden evidente en historial.	Repositorio incompleto, desorganizado o sin commits válidos.	3%
IE 2.5.2 Organiza tareas del proyecto en Trello u otra herramienta colaborativa, evidenciando roles y avance del equipo.	Tablero organizado con tareas claras, asignaciones y avance visible.		Tablero con organización parcial.	Tablero incompleto o sin asignaciones reales.	No utiliza planificación.	2%
IE 3.1.1 Desarrolla pruebas unitarias para validar la lógica del microservicio, aplicando correctamente frameworks de testing y asegurando cobertura mínima	Desarrolla pruebas unitarias con al menos un 80% de cobertura, implementa flujos Given–When–Then, utiliza asserts precisos; cubre reglas de negocio esenciales y obtiene resultados estables.		Desarrolla pruebas unitarias con menos del 80% de cobertura, implementa flujos Given–When–Then, utiliza asserts precisos; cubre reglas de negocio esenciales y obtiene resultados estables.	Desarrolla pruebas unitarias con menos del 80% de cobertura, pero inconsistentes o con errores recurrentes.	No desarrolla pruebas válidas o estas no funcionan.	8%

sobre funciones y reglas de negocio.						
IE 3.2.1 Elabora documentación técnica clara, estructurada y trazable utilizando Swagger y OpenAPI, incluyendo descripciones de endpoints, ejemplos, códigos de respuesta y modelos de datos	Documenta endpoints, parámetros, modelos, ejemplos y códigos HTTP con precisión; la documentación coincide completamente con el comportamiento real.		Documentación funcional con omisiones menores o falta de estandarización.	Documentación incompleta, incoherente o parcialmente funcional.	No existe documentación válida o se genera muy poca documentación.	4%
IE 3.3.1 Configura correctamente el despliegue del microservicio en entornos locales o remotos (Docker, Railway, Render u otros), asegurando que la aplicación opere sin errores.	Configura y despliega los microservicios en al menos dos de las herramientas solicitadas: Docker, Railway, Render u otros; variables, puertos y parámetros correctos; servicio estable y operativo.		Configura y despliega los microservicios en una de las herramientas solicitadas: Docker, Railway, Render u otros; variables, puertos y parámetros correctos; servicio estable y operativo.	Despliegue incompleto o con errores recurrentes en las dos herramientas solicitadas.	No logra desplegar los microservicios o lo hace en una sola herramienta con errores recurrentes.	5%
IE 3.3.2 Configura un API Gateway (Spring Cloud Gateway u equivalente) para centralizar y administrar el enrutamiento entre microservicios, garantizando coherencia y control del flujo de solicitudes en la arquitectura distribuida.	Configura rutas, predicados, filtros y mapeos con precisión; el enrutamiento opera de forma estable y coherente.		Gateway funcional, pero con inconsistencias menores.	Rutas incompletas, errores frecuentes o manejo deficiente de solicitudes.	No existe enrutamiento funcional a través del Gateway.	5%

IE 3.3.3 Gestiona la interoperabilidad entre los servicios mediante un diseño adecuado de rutas, filtros y reglas de enrutamiento aplicado en el Gateway.	Define reglas de enrutamiento consistentes; valida flujos; asegura comunicación estable y consistente entre servicios.		Interoperabilidad funcional con omisiones menores.	Rutas o filtros inconsistentes que afectan comportamiento.	Fallas críticas en la interoperabilidad.	2%
IE 3.3.4 Configura archivos YAML para manejar propiedades de entorno, perfiles, rutas del Gateway, variables de entorno, puertos y configuraciones relevantes de los microservicios.	Configura archivos YAML limpios, organizados, coherentes con el despliegue y completos en rutas, perfiles y propiedades.		YAML funcional con pequeñas inconsistencias.	Configuración incompleta desordenada. o	No configura YAML de manera válida.	3%
Porcentaje Entrega de Encargo						40%
Dimensión Defensa						
IE 2.2.2 Explica con claridad el funcionamiento del código desarrollado en el proyecto, justificando decisiones técnicas y relacionándolas con el dominio de los microservicios.	Explica adecuadamente relaciones, lógica, responsabilidades y decisiones de diseño; identifica impactos en el servicio; demuestra comprensión total del código.		Explica parcialmente relaciones, lógica, responsabilidades y decisiones de diseño; identifica impactos en el servicio, pero algunas decisiones no están justificadas o muestran vacíos conceptuales.	Explica superficialmente sin claridad sobre relaciones, lógica, responsabilidades y decisiones de diseño.	No identifica correctamente la lógica ni las decisiones implementadas.	5%
IE 2.4.2 Explica consistencia de datos e interoperabilidad mediante diseño adecuado de endpoints, DTOs y	Explica de manera correcta y exacta el flujo remoto, diseño de endpoints, DTOs, validaciones y coherencia entre servicios.		Explica el flujo remoto, diseño de endpoints, DTOs, validaciones y coherencia entre servicios, pero con omisiones menores.	Faltan elementos clave o confunde modelos/DTOs.	No comprende el consumo remoto ni la interoperabilidad.	5%

consumo del servicio remoto						
IE 2.5.3 Explica su aporte personal al proyecto, justificando commits, tareas asignadas y participación individual en el desarrollo del proyecto semestral.	Presenta claridad total sobre su trabajo, commits, roles y aportes individuales.		Explica parcialmente su participación.	Explicación vaga o poco estructurada.	No puede explicar su aporte.	5%
IE 3.1.2 Explica con claridad el funcionamiento de las pruebas unitarias generadas en el proyecto, justificando decisiones técnicas y relacionándolas con el dominio del microservicio.	Explica con claridad la estructura de las pruebas unitarias implementadas, la función evaluada y el propósito de cada assert; describe correctamente el uso de mocks, la simulación del repositorio y la razón de cada comportamiento configurado; identifica qué regla de negocio se está validando y justifica por qué se eligieron esos casos; interpreta los resultados y explica cómo la prueba garantiza el correcto funcionamiento del servicio.		Explica las pruebas unitarias implementadas y algunos asserts; reconoce el uso de mocks o simulaciones, pero con justificaciones incompletas; identifica parcialmente la regla de negocio evaluada; interpreta el resultado general de la prueba, aunque con vacíos en la explicación fina del flujo interno	Describe aspectos aislados de las pruebas unitarias implementadas sin profundizar en su propósito; confunde mocks, asserts o la estructura Given–When–Then; no relaciona la prueba con la regla de negocio evaluada; proporciona una explicación poco técnica o incompleta	No logra describir la estructura, propósito o funcionamiento de las pruebas unitarias implementadas; desconoce mocks, asserts o la lógica interna; no puede relacionar la prueba con el servicio ni interpretar su resultado	7%
IE 3.1.3 Integra nuevas pruebas unitarias en tiempo acotado, aplicando correctamente la lógica requerida y manteniendo	Implementa una nueva prueba completamente funcional; estructura Given–When–Then correctamente; configura mocks precisos y produce asserts coherentes con la lógica solicitada; la		Implementa una prueba funcional, pero requiere ajustes menores para la correcta simulación del repositorio o para que los asserts reflejen el resultado esperado; corrige los errores tras	Implementa parte de la prueba, pero presenta errores persistentes en mocks, asserts o estructura; no logra alinearse a la lógica solicitada; la prueba no ejecuta correctamente	No logra implementar la prueba; no sabe qué simular ni qué evaluar; provoca errores críticos o no puede ejecutar el test.	13%

consistencia del proyecto.	prueba compila y se ejecuta sin errores; demuestra capacidad técnica y seguridad al modificar código en tiempo real.		indicación mínima del docente y logra obtener un resultado exitoso.	o no refleja el comportamiento esperado del servicio.		
IE 3.2.2 Explica la documentación construida detallando cómo se usa el microservicio	Recorre la interfaz mostrando rutas, parámetros, modelos, request bodies y códigos de respuesta; explica cómo consumir cada endpoint, qué envía y qué recibe; interpreta correctamente los ejemplos generados; relaciona la documentación con la lógica real del servicio y demuestra dominio total del flujo de uso del microservicio.		Describe rutas principales, parámetros y algunos modelos; explica el consumo básico del servicio, pero con omisiones en códigos de respuesta, ejemplos o vínculos entre documentación y lógica interna.	Menciona elementos aislados de la documentación, pero sin explicar su propósito; confunde modelos, parámetros o rutas; no puede indicar cómo se consume correctamente el servicio.	No identifica rutas, modelos ni parámetros; no logra explicar cómo usar el microservicio mediante la documentación.	5%
IE 3.3.5 Explica la configuración YAML utilizada en el proyecto y justifica su aplicación en el despliegue.	Describe propiedades clave, la función de cada sección del archivo; distingue entre configuraciones locales y remotas; interpreta cómo YAML afecta el despliegue, comunicación entre servicios y comportamiento de la aplicación.		Reconoce propiedades principales, explica conceptos generales, pero sin profundidad o sin relacionarlos completamente con la operación del sistema.	Confunde propiedades, perfiles o rutas; identifica parámetros sin explicar su función; proporciona una lectura descriptiva sin análisis técnico.	No entiende la configuración YAML; no identifica propiedades ni comprende su impacto en la aplicación.	4%
IE 3.3.6 Describe el proceso de despliegue utilizado, detallando	Describe plataforma utilizada (Docker, Railway, Render u otra),		Describe el proceso general de despliegue y funcionamiento, pero	Proporciona un relato incompleto; confunde etapas del despliegue;	No comprende el proceso de despliegue ni cómo	6%

entornos, configuraciones, puertos y funcionamiento final del microservicio o ecosistema distribuido.	variables configuradas, archivos necesarios, puertos expuestos, logs, arranque del contenedor o servicio, rutas generadas y funcionamiento final del microservicio; interpreta errores de despliegue y explica cómo fueron resueltos.		omite detalles técnicos como variables, puertos o logs; entiende el flujo principal, pero sin profundidad.	no explica la relación entre configuración y ejecución final.	el servicio llega a ejecutarse; desconoce plataforma, configuración o funcionamiento.	
IE 3.3.7 Realiza la configuración de manera individual y sin apoyo para ejecutar los microservicios en entornos locales y remotos de acuerdo a lo que hayan construido como equipo.	Configura y ejecuta correctamente los servicios, ajusta puertos, perfiles, variables o rutas de Gateway sin asistencia; identifica problemas de ejecución y aplica correcciones rápidas; demuestra dominio completo del ecosistema.		Ejecuta los microservicios con asistencia mínima; requiere ajustes menores indicados por el docente; logra levantar el sistema tras correcciones.	Realiza parte del proceso, pero comete errores que impiden el funcionamiento; presenta dificultades en YAML, puertos, rutas o ejecución de servicios.	No logra configurar ni ejecutar los servicios; no entiende los pasos ni cómo resolver errores.	10%
Porcentaje Defensa						60%
Total						100%